

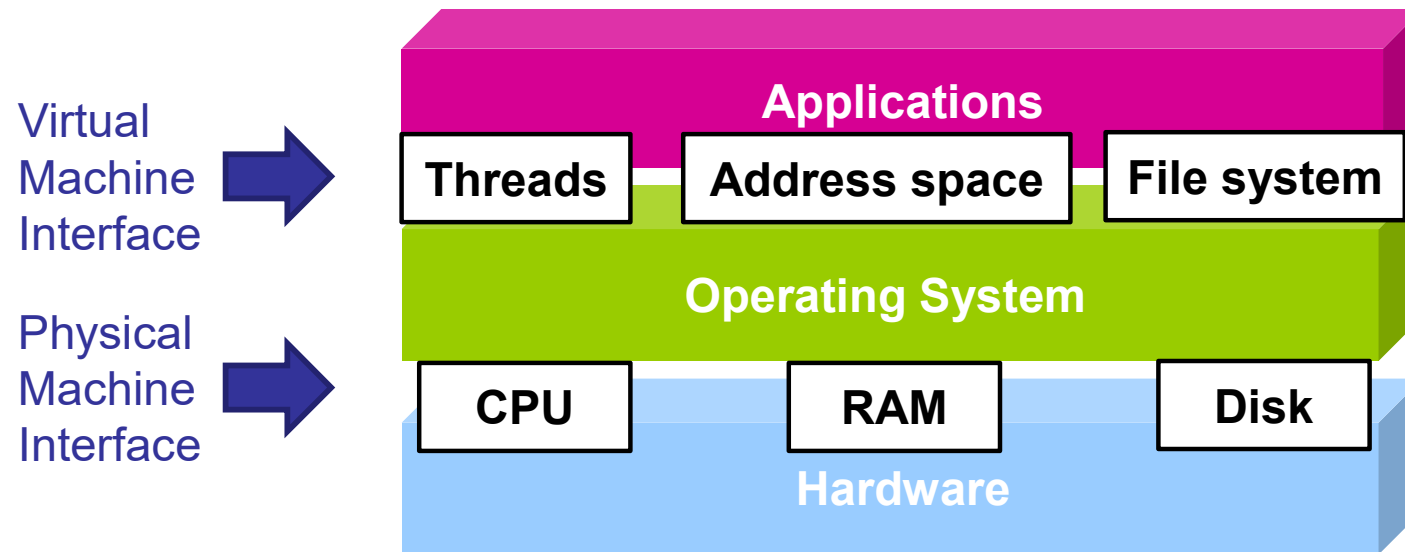
EECS 482

Introduction to operating systems

File and storage systems

Peter Chen
University of Michigan

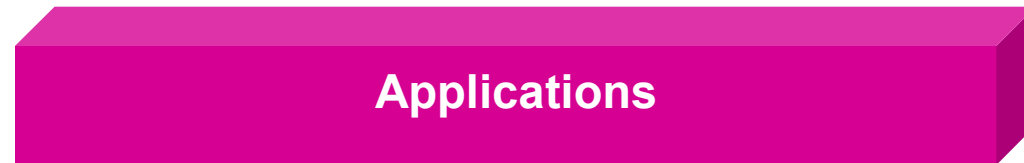
OS abstractions



File system abstractions

Hardware reality	OS abstraction
Heterogenous interfaces	Uniform interface
A few storage objects (disks)	Many storage objects (files)
Simple names (numeric)	Rich naming structure (symbolic, hierarchical, unified)
Slow	Fast
Inconsistent on crash	Consistent on crash

Dealing with heterogeneity



Applications



Device Drivers

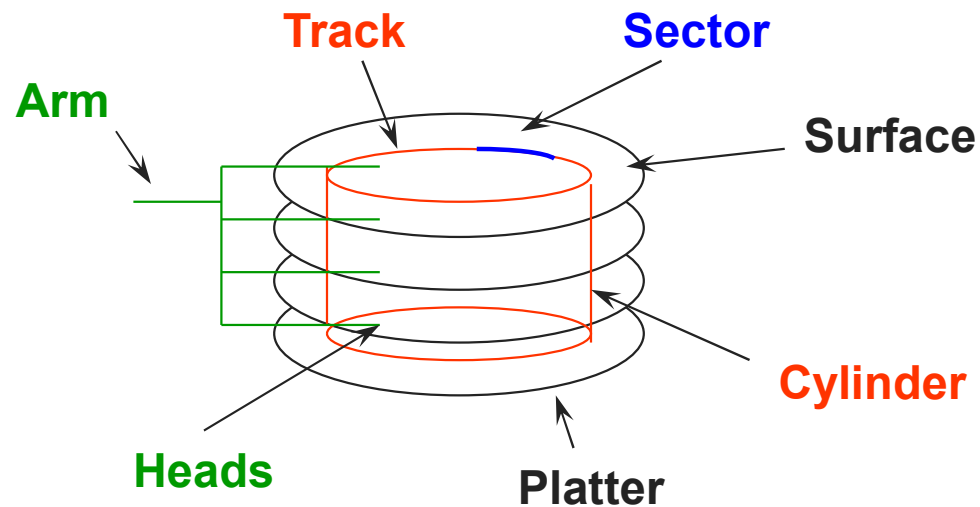
Uniform interface
File systems
to storage (and other) devices



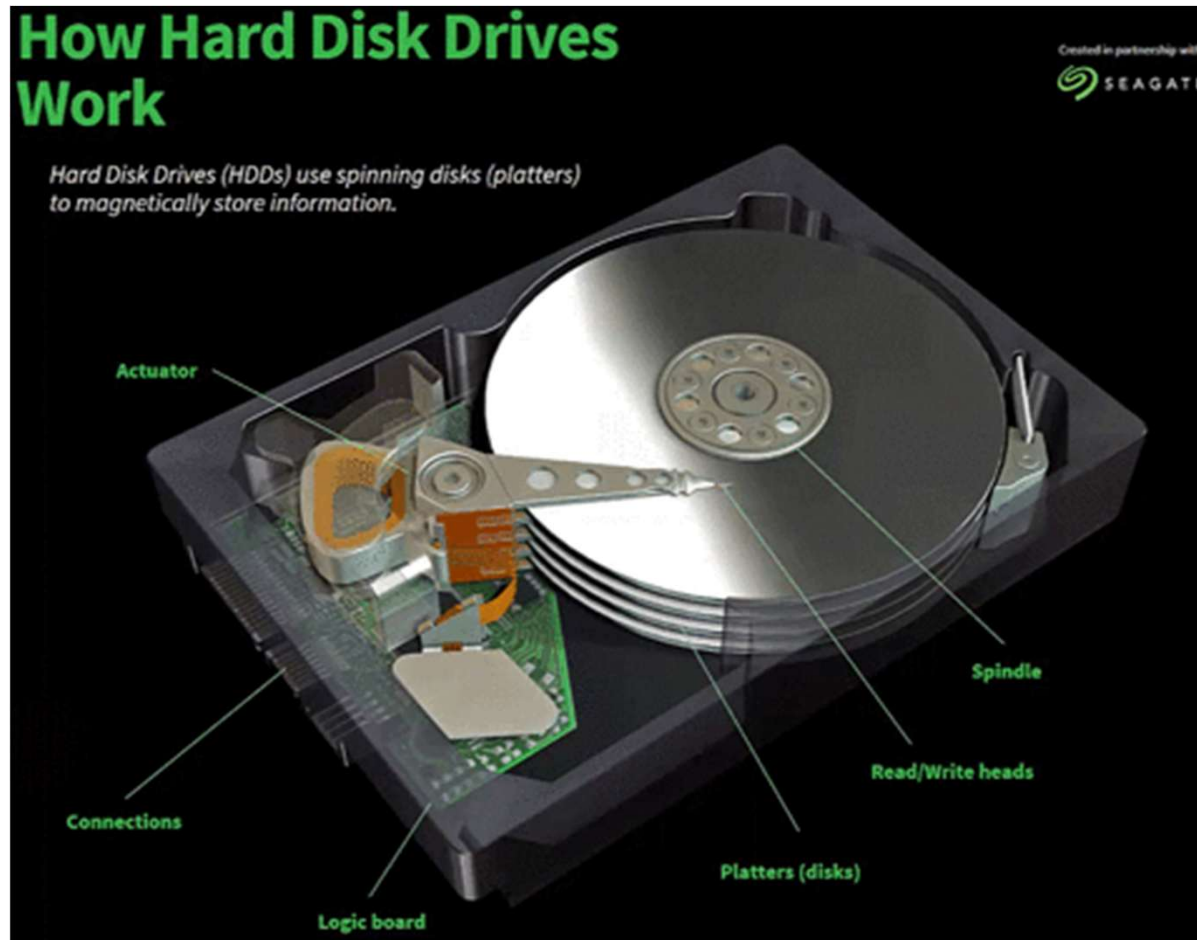
Hardware

Heterogeneous storage
devices and interfaces

Physical hard disk structure



Magnetic hard drive



Time to access storage device

- Queue: wait for the device to be free
- Overhead
 - E.g., position disk head and arm (often a few milliseconds)
- Data transfer: transfer data from/to device
 - size / transfer rate
 - transfer rate is often a few hundreds of MB/s

Optimizing I/O performance

- General strategies
 - Avoid doing I/O (disks are slow!)
 - Reduce overhead (minimize positioning time)
 - Amortize overhead over larger requests
- How to avoid doing I/O?
- How to reduce positioning time?

Disk scheduling

- Reduce overhead by reordering requests
 - First-come, first-served (FIFO)
 - Shortest seek time first
 - » Reduces response time
 - » Problems?
 - SCAN (a.k.a. elevator algorithms)
 - » Better throughput (minimizes total disk head movement)
- Disk scheduling vs. CPU scheduling
 - Does CPU scheduling affect throughput?
 - Does disk scheduling affect throughput?

Other ways to reduce positioning time?

Amortizing overhead

- Efficiency = transfer time / (overhead + transfer time)
- E.g., to achieve 50% efficiency, overhead should equal transfer time

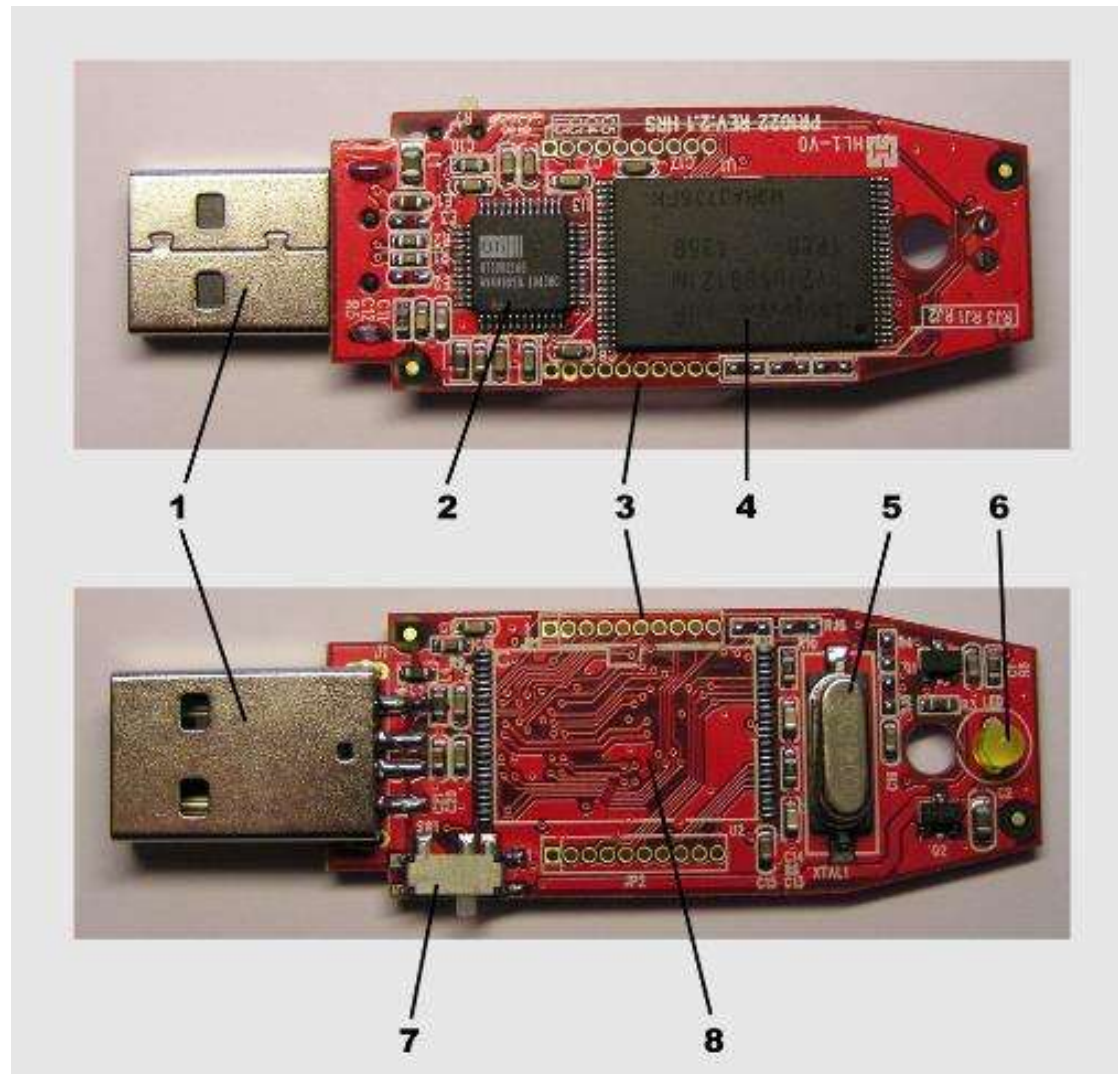
overhead = transfer time

→ overhead = size / transfer rate

→ size = overhead * transfer rate. In networking, this is called the **bandwidth-delay product**

E.g., 8 ms overhead, 100 MB/s transfer rate → 800 KB transfer to achieve 50% efficiency

USB flash drive



Solid state disks (e.g., flash memory)

- Optimizations depend on specifics of a device
- Flash differs from magnetic disk
 - + Better random read performance
 - + Lower positioning overhead
 - + Lower power
 - + Better shock resistance
 - Wearout
 - Cannot overwrite data in place

Optimizing for flash

File abstraction

- Hardware reality: a few storage objects (disks)
- OS abstraction: numerous storage objects (files)
 - Created and destroyed on demand
 - Named and organized for user convenience
- Challenges:
 - How to name files?
 - How to find and organize files?
 - How to keep file data consistent in the presence of crashes?

File systems

- File system: a **persistent** data structure
- Persistent across what?
 - Process creation/exit
 - Machine crashes/reboots
 - Power outages
- How to enable persistence across these events?
 - Use persistent storage medium
 - Use persistent pointers (addresses that are stable across reboot), e.g., disk block number
 - Write data in a careful order

Interface to file system

- Create file
- Delete file
- Read <file, offset>
- Write <file, offset>
- Other (e.g., list files in a directory)
- Other persistent data structures
 - SQL database
 - Key-value store

File system workloads

- Optimize data structure for the common case
- Some general rules of thumb
 - Most file accesses are reads
 - Most programs access files sequentially and entirely
 - Most files are small, but most bytes belong to large files